

FINAL DEVELOPMENT ROADMAP

Portable 32-bit RISC-V Processor

From the current RV32I subset pipeline to a verified
RV32IM_Zicsr_Zifencei bare-metal core

TARGET	PORTABILITY
5-stage in-order core Machine mode and precise traps Optional multi-cycle M extension	Board-neutral CPU and SoC Replaceable Xilinx wrappers No physical board required for core completion

Prepared for the Simple RISC Processor repository
24 August 2026

1. Final scope and definition of done

The project will produce a reusable 32-bit RISC-V processor and a small bare-metal system, not an implementation of every optional RISC-V extension. The chosen endpoint is ambitious enough to run compiled C firmware and handle real interrupts, while remaining realistic for a compact FPGA project.

FINAL TARGET: RV32IM_Zicsr_Zifencei, Machine mode, precise synchronous exceptions, machine interrupts, generic instruction/data interfaces, and a portable bare-metal SoC.

Core-complete acceptance criteria

- All required RV32I instructions execute with architecturally correct results.
- The M extension implements multiply, divide, and remainder operations, including mandated corner cases.
- CSR instructions, Machine-mode traps, interrupts, and MRET behave precisely.
- No younger or faulting instruction can commit an incorrect register or memory side effect.
- The core tolerates variable-latency instruction and data responses.
- Directed, hazard, randomized-latency, and official architecture tests pass.
- The generic RTL synthesizes cleanly and a representative Xilinx implementation meets its declared clock constraint.

System-complete acceptance criteria

- A generic SoC integrates boot memory, RAM, UART, GPIO, and a machine timer.
- A cross-compiled C program boots, prints through UART, services a timer interrupt, and controls GPIO.
- Board-specific details are confined to wrappers, constraints, clock/reset blocks, and optional memory adapters.

Explicit non-goals for this release

Not required	Reason
Linux boot	Requires Supervisor mode, an MMU, much more memory, and broader platform support.
Caches and branch prediction	Performance features are separable from ISA correctness and portability.
A/F/D/C/V extensions	Optional follow-on work after the base core is stable and compliant.
One universal FPGA top module	Pins, clocks, reset, memories, and peripherals necessarily vary by board.

2. Starting point and target architecture

Current repository baseline

Area	Current capability	Primary gap
Pipeline	Five stages with forwarding, load-use stall, and taken-branch flush	Unified control for jumps, traps, interrupts, and memory stalls
ISA	ADD/SUB, logic, immediate logic, LW/SW, BEQ	Most RV32I instructions, SYSTEM behavior, and M extension
Memory	Fixed 64-word instruction ROM and data RAM	Byte lanes, access size, faults, and variable latency
Verification	Six self-checking directed machine-code tests	Assembler flow, reference comparison, assertions, and architecture tests
Platform	Basys 3 wrapper and LED MMIO	Generic SoC boundary and per-board wrappers

Required portability boundary

PORTABLE AND BOARD-NEUTRAL
RISC-V CPU core
Generic instruction/data request-response interfaces
Generic SoC: address decoder, ROM/RAM, UART, timer, GPIO
BOARD-SPECIFIC
Xilinx wrapper: clock/reset, optional Xilinx IP and memory adapters
Vivado part selection, XDC pins, timing constraints, and build script

The CPU must not instantiate Xilinx clocking, BRAM, UART, DDR, or board I/O primitives. Portable inferred memories may be supplied as default adapters, while vendor IP remains outside the core.

3. Implementation roadmap - foundation and RV32I

0

Freeze the architectural contract

Remove ambiguity before changing the pipeline.

- Document ISA string, little-endian behavior, reset vector, instruction alignment, and memory map.
- Choose trap-on-misaligned-access behavior and define illegal-instruction handling.
- Specify request/response timing, byte masks, error response, and cancellation rules.
- Record the retirement event format used by all later verification.

Exit gate: a concise architecture specification is reviewed and test expectations are unambiguous.

1

Build the verification foundation

Make architectural mistakes observable before expanding functionality.

- Add a retirement interface: valid, PC, instruction, register write, memory effect, and trap metadata.
- Compile assembly tests into ELF/HEX instead of maintaining machine code manually.
- Retain the six existing regressions and add automatic timeout and failure diagnostics.
- Add invariants for x0, wrong-path suppression, control exclusivity, and in-order retirement.
- Create a path to compare traces or signatures against Sail or Spike.

Exit gate: all existing tests pass through the new flow, with a stable retirement trace.

2

Complete RV32I

Finish the base ISA in independently testable instruction families.

Batch	Instructions / behavior	Pipeline impact
2A	LUI, AUIPC, JAL, JALR	PC+4 writeback, target generation, control flush
2B	SLT/U, SLL, SRL, SRA and immediate forms	Signed comparisons and shift amount rules
2C	BNE, BLT, BGE, BLTU, BGEU	Dedicated signed/unsigned branch comparison
2D	LB/U, LH/U, SB, SH	Byte enables and load extension in the data path
2E	FENCE, FENCE.I, ECALL, EBREAK, illegal encodings	Trap generation and defined instruction-fetch synchronization

Decoder rule: every encoding must either produce a complete, valid control record or an illegal-instruction trap.

Unsupported funct3/funct7 combinations must never default to ADD. FENCE.I must synchronize later instruction fetches with earlier stores; a simple implementation may flush/refetch because it has no instruction cache.

Exit gate: directed and applicable official RV32I architecture tests pass.

4. Implementation roadmap - memory, traps, and M

3

Introduce scalable memory interfaces

Decouple the pipeline from fixed, zero-latency arrays.

- Expose separate instruction and data request/response channels with address, write data, byte mask, and error status.
- Freeze the correct pipeline stages while requests are outstanding; prevent duplicate stores.
- Define priority among reset, trap, interrupt, redirect, memory stall, and load-use stall.
- Provide simulation RAM and synthesizable inferred-BRAM adapters.
- Test zero-latency, fixed-latency, and randomized-latency memories with identical expected retirement traces.

Exit gate: randomized memory latency cannot alter architectural results.

4

Add Zicsr and Machine mode

Make exceptions and interrupts precise and firmware-usable.

Capability	Required content
CSR instructions	CSRRW/RS/RC and immediate variants, privilege and read-only checks
Core CSRs	mstatus, misa, mie, mip, mtvec, mscratch, mepc, mcause, mtval
Counters	mcycle and minstret with clearly defined widths and access behavior
Synchronous traps	Illegal instruction, breakpoint, ECALL, instruction/data misalignment and access faults
Interrupts	Machine software, timer, and external interrupt inputs with documented priority
Return	MRET restores execution without committing flushed instructions

Precision rule: the faulting instruction and all younger instructions must be prevented from producing unintended side effects; mepc, mcause, and mtval must identify the architectural event correctly.

Exit gate: a firmware trap handler can distinguish causes, update mepc when appropriate, and resume through MRET.

5

Add the M extension

Add multi-cycle arithmetic without weakening pipeline correctness.

- Implement MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, and REMU.
- Start with an iterative, multi-cycle unit to conserve FPGA resources.
- Hold EX and upstream state while the unit is busy, while allowing older instructions to drain safely.
- Cover division by zero, signed overflow, sign combinations, and destination dependencies.

Exit gate: all applicable RV32M architecture tests pass and the representative Xilinx target still meets timing.

5. Implementation roadmap - SoC, portability, and release

6

Build the generic bare-metal SoC

Prove the core can execute normal compiled firmware.

- Integrate boot ROM, RAM, UART, GPIO, and a machine timer behind a documented address decoder.
- Use aligned MMIO regions; remove the current special LED address at 0xFFFFFFFF.
- Provide startup assembly, linker script, C build flow, and minimal runtime support.
- Demonstrate UART output, timer interrupts, GPIO control, and exception reporting.

Exit gate: a cross-compiled C firmware image boots in simulation and completes all peripheral demonstrations.

7

Harden, synthesize, and release

Turn a working design into a reproducible processor project.

- Run lint, directed tests, architecture tests, randomized-latency tests, and synthesis in CI.
- Measure CPI, LUTs, registers, BRAM, DSP usage, and achieved clock frequency.
- Test reset during stalls and multi-cycle operations, plus interrupt/trap boundary cases.
- Document the ISA profile, implementation-defined behavior, address map, bus protocol, and integration guide.
- Publish a known-good generic simulation configuration and at least one Xilinx synthesis configuration.

Exit gate: the repository can be cloned, tested, synthesized, and used to run the example firmware from documented commands.

Xilinx board transfer procedure

Usually unchanged	Changed for each board	Sometimes required
CPU RTL CSR/trap logic Generic bus Generic peripherals Firmware ABI	Top-level wrapper FPGA part number XDC pin assignments Clock/reset setup Available I/O	BRAM adapter Clocking wizard DDR controller and bridge Vendor-specific IP Timing retuning

A physical FPGA board is not required to complete or verify the CPU. Board ownership becomes important only for validating electrical I/O, the actual clock/reset circuit, external memories, and physical peripherals.

6. Recommended work-package sequence

Each package should end with a green regression and a documented interface. Avoid combining decoder expansion, memory refactoring, and trap handling in one large change.

ID	Package	Primary output	Depends on
WP01	Architecture contract	Spec, memory map, bus, retirement record	All later work
WP02	Retirement and assembly flow	Commit trace, ELF/HEX generation, retained tests	WP01
WP03	Decoder/control refactor	Typed control record and illegal decode	WP02
WP04	ALU and upper-immediate completion	RV32I arithmetic, shifts, LUI/AUIPC	WP03
WP05	Jumps and branches	JAL/JALR and all branch conditions	WP04
WP06	Subword memory	LB/LH variants, SB/SH, byte masks	WP03
WP07	Memory handshake and stalls	Variable latency and error response	WP05-WP06
WP08	Precise synchronous traps	Illegal, ECALL/EBREAK, alignment, access faults	WP07
WP09	CSRs and MRET	Zicsr and Machine trap state	WP08
WP10	Interrupts	Software, timer, external; priority and masking	WP09
WP11	M extension	Iterative multiply/divide unit	WP07, WP09
WP12	Generic SoC and firmware	ROM/RAM/UART/GPIO/timer and C examples	WP10-WP11
WP13	Compliance and release	Official tests, CI, synthesis, docs	All

Recommended first implementation move: complete WP01 and WP02 before adding instructions. The retirement trace and assembler-driven tests will pay for themselves throughout every later phase.

7. Verification strategy and project controls

Verification ladder

Level	Purpose	Required evidence
Module	Validate combinational and sequential building blocks	Decoder, ALU, CSR, divider, and bus adapter unit tests
Instruction	Validate each architectural operation	Positive, negative, boundary, and aliasing cases
Pipeline	Validate interactions	RAW dependencies, load-use, redirects, stalls, traps, interrupts
Reference	Detect architectural divergence	Retirement trace or signature comparison to Sail/Spike
Compliance	Exercise specification rules	Applicable official RISC-V architecture tests
Implementation	Validate FPGA feasibility	Vivado synthesis, timing, utilization, and later board smoke test

Primary risks and mitigations

Risk	Control
Pipeline control becomes ad hoc	Centralize event priority and carry valid bits through every pipeline stage.
Wrong-path or faulting stores commit	Gate side effects with stage-valid and retirement/commit conditions; assert the property.
Board assumptions leak into the CPU	Enforce module boundaries and review dependencies before every merge.
Architecture tests are added too late	Build the adapter during RV32I completion, then grow coverage continuously.
DDR integration dominates the project	Use on-chip RAM first; add DDR only after the core and generic SoC are complete.
Timing regresses after new controls	Run periodic synthesis and register or simplify long redirect/stall paths early.

8. Final completion checklist

Architecture ☐ RV32I complete ☐ RV32M complete ☐ Zicsr complete ☐ Zifencei behavior defined ☐ Machine mode and MRET complete	Correctness ☐ Precise synchronous traps ☐ Software/timer/external interrupts ☐ Misaligned behavior verified ☐ Variable-latency memory verified ☐ No wrong-path side effects
Platform ☐ Generic bus documented ☐ ROM/RAM/UART/GPIO/timer integrated ☐ C firmware demonstrations pass ☐ Board-specific logic isolated ☐ Representative Xilinx synthesis passes	Release ☐ Automated regression ☐ Architecture tests recorded ☐ Timing/utilization recorded ☐ Integration guide complete ☐ Known limitations published

Decision summary: finish and verify the portable processor without waiting for a board choice. When a Xilinx board is selected, create a thin wrapper, constraints, and any required memory adapter; do not redesign the CPU.

Authoritative references

- RISC-V RV32I Base Integer Instruction Set, Version 2.1: <https://docs.riscv.org/reference/isa/unpriv/rv32.html>
- RISC-V Control and Status Registers: <https://docs.riscv.org/reference/isa/priv/priv-csrs.html>
- RISC-V Machine-Level ISA, Version 1.13: <https://docs.riscv.org/reference/isa/priv/machine.html>
- RISC-V Architectural Certification Tests: <https://github.com/riscv/riscv-arch-test>

This roadmap is based on the repository state reviewed on 24 August 2026. It intentionally separates ISA correctness from board integration so development can proceed without physical FPGA hardware.